

Q1 Trace the processing of the expression $P = (a * b) + (b * c) * 2$

a) Lexical Analysis (Token Generation)

⇒ The lexical analyzer scans the input character and groups them into tokens.

Lexeme	Token Type
P	identifier
=	assignment operator
(	left parenthesis
a	identifier
*	multiplication operator
b	identifier
)	right parenthesis
+	addition operator
(	left parenthesis
b	identifier
*	multiplication operator
c	identifier
)	
*	multiplication operator
2	integer constant

Total tokens = 15

⑥ Syntax Tree



① Root node is assignment (=).

② Left child is variable r.

⑦ Intermediate Code

* $t_1 = a * b$

* $t_2 = b * c$

* $t_3 = t_2 * 2$

* $t_4 = t_1 + t_3$

* $r = t_4$

⑧ Code Optimisation

* possible optimisations

* Constant folding

* NO constant Expression exists except multiplication by 2.

* strength reduction

{ $\Rightarrow t_3 = t_2 * 2$ }

Qn
Q Construct the language and write five valid string.

a) $(aa^*)^*$

* since $\Rightarrow aa^*$

* means $\Rightarrow a^+$

* and $\Rightarrow (a^+)^*$

* means any number of a's including empty string

language

* $L = \{ \epsilon, a, aa, aaa, aaaa, \dots \}$

five string

ϵ

a

aa

aaa

aaaa

b) $(b^*)^* abb$

* since:

$$(b^*)^* = b^*$$

* Expression becomes

$b^* abb$

language

any number of b's followed by abb

Optimised Code:-

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 \ll 1$

$p = t_1 + t_3$

ex Final Target Code:-

Eg assembly:

LOAD R1, a

MUL R1, b

LOAD R2, b

MUL R2, c

SHL R2, 1

ADD R1, R2

STORE P, R1.

Significance:-

- Machine executable code
- faster execution
- less memory usage
- Optimised instructions

Qn 3 (i) Find number of tokens

* Program:

```
main()
```

```
{
```

```
int a = 10;
```

```
char b = "abc";
```

```
int c = 30;
```

```
char d = "xyz";
```

```
int m = 40.5%;
```

```
}
```

→ Comments are ignored

→ total tokens = 33

(ii) Lexical error:-

```
void main()
```

```
{
```

```
int x = 10, y = 20;
```

```
char * a;
```

```
a = 5x;
```

```
x = 1xab;
```

```
}
```

error 1

1xab

invalid numerical literal

error 2

a = 5x; lexically correct

dr $(ab)^*$

* two or more repetitions of "ab"

* language

$$\{ (ab)^n \mid n \geq 0 \}$$

* Example:

ϵ

ab

ab ab

ab ab ab

ab ab ab ab

ex $(aba)^* + (ab)^*$
w

* union of

* Repeated "aba"

(OR)

* any number of a's followed by b's

eg:

ab a

ab a ab a

a a d b b b

a a b

b b b

Qn
④

(i) Even number of 0's

Regular Expression

(i) Even number of 0's

→ regular expression

* $(0^* (01^* 01^*)^*)^*$

→ explanation

* zeroes always occur in pairs

ii) At least one 1

* $(0^* 1)^* (0^* 1)^*$

contains one or more 1

→ Eg:

1

0 1

1 0 0

1 1 1

1 0 1 0

(iii) odd number of 0's and odd number of 1's

* one valid expression is

→ $(00+11)^* (01+10) (00+11)^*$

* this ensure both counts become odd

* Eg:

01

10

00011

11000

101

(iv) start with 1 and end with 0

⇒ $(10+1) * 0$

* Eg:

10

100

110

1010

1110

(v) No consecutive 1's

⇒ $(10+10) * (1+E)$

Eg:

0

1

010

1010

10010

an
⑤

input:-

int var123 = 12345;

float var456 = 123.45;

if (var123 == var456)

{

return;

}

⇒ The lexical analyser classifies every token into its corresponding type

a) No. of. tokens

* token type

* int keyword

* var123 identifier

* = operator

* 12345 integer literal

* ; separator

* float keyword

* var 456 identifier

* 123.45 float literal

* ; separator

* if keyword

* { separator

⇒ Total tokens = 20

b) Categorised tokens

⇒ keyword

int

float

if

return

* Operator

=

=

==

* Separator

;

;

(

)

{

;

}

* Identifier

var 123

var 456

Var 123

vor 456

Cy
m

FSM State Transition

⇒ Typical DFA Transition

⇒ Keyword

start → i → n → t → final

≈ 4 transitions

⇒ Identifier

start → letter

→ letter → final

→ letter

→ digit

→ digit